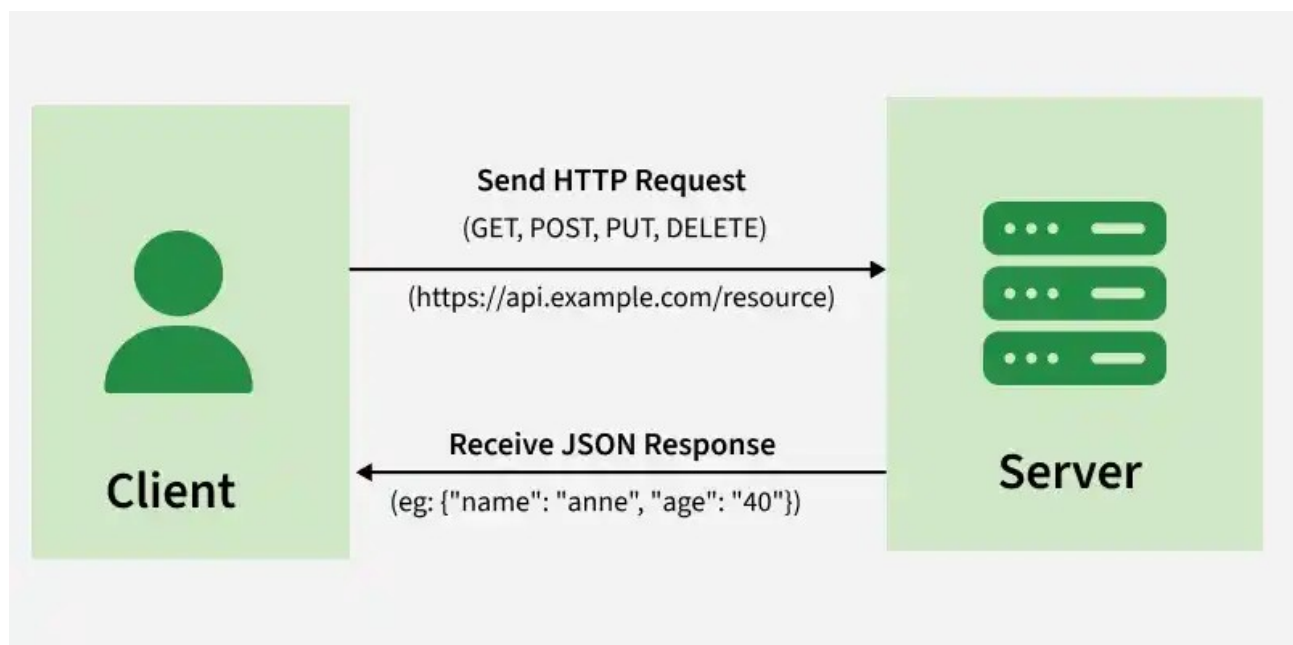


REST API Introduction

A REST API (Representational State Transfer API) is a set of rules and protocols that enables communication between client and server over HTTP. It exchanges data typically in JSON format using standard web protocols.

- Uses HTTP methods like GET, POST, PUT, PATCH, and DELETE.
- Client sends requests to server endpoints (URLs).
- Server returns responses such as JSON, XML, HTML, or images.
- Maps HTTP methods to CRUD operations (Create, Read, Update, Delete).



1. HTTP

HTTP (HyperText Transfer Protocol) is the communication protocol used between a **client** and **server**.

we use HTTP to define how a request is sent and how the server responds.

Example

Frontend → HTTP Request → Backend API
Frontend ← HTTP Response ← Backend API

For example, when a frontend wants employee data:

```
GET /api/employees
```

The backend processes the request and returns employee data.

2. Request

An **HTTP request** is a message sent by the client to the server to perform an operation.

A request can contain:

- **HTTP Method** – what operation to perform
- **URL** – which resource to access
- **Headers** – additional information
- **Query/Path parameters** – additional values
- **Request Body** – data sent to the server

Example

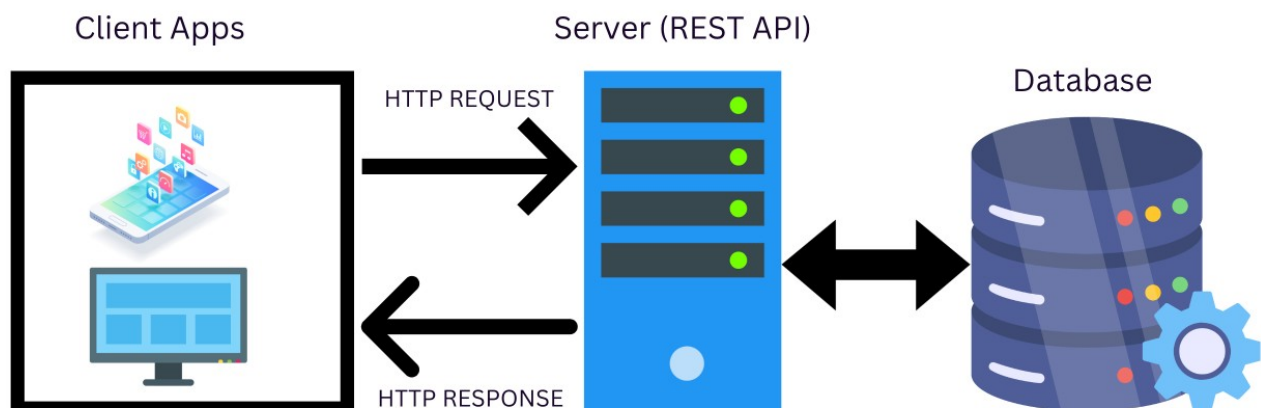
```
POST /api/employees  
Content-Type: application/json
```

```
{  
  "name": "Rohit",  
  "email": "rohit@example.com",  
  "department": "IT"  
}
```

Here, the client is asking the server to create an employee.



WWW.QASCRIPT.COM



3. Response

An **HTTP response** is the server's answer to the client's request.

A response normally contains:

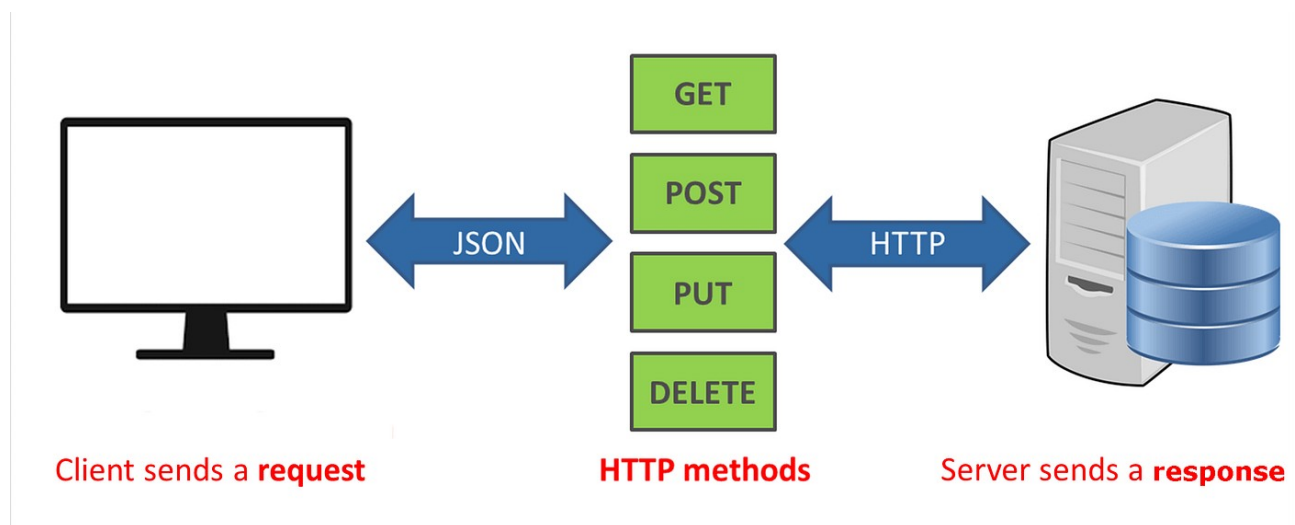
- **Status code**
- **Headers**
- **Response body**

Example

```
HTTP/1.1 201 Created
Content-Type: application/json
```

```
{
  "id": 101,
  "name": "Rohit",
  "email": "rohit@example.com",
  "department": "IT"
}
```

The 201 tells the client that the employee was successfully created.



4. JSON

JSON (JavaScript Object Notation) is a lightweight format commonly used to exchange data between frontend and backend.

It represents data using **key-value pairs**.

Example

```
{
  "id": 101,
  "name": "Rohit",
}
```

```
{  "department": "IT",
  "skills": ["Python", "SQL", "Django"]}
}
```

As a backend developer, we will frequently:

- Receive JSON from the client.
- Validate the JSON.
- Store its data in a database.
- Return JSON in the response.

Why is JSON used?

- **Easy to read and understand** by developers.
- **Lightweight**, so it is efficient for network communication.
- **Easy to parse and process** by programming languages.
- **Language-independent** — works with Python, Java, JavaScript, C++, etc.
- **Supports structured data** like objects, arrays, strings, numbers, and booleans.
- **Commonly used in REST APIs** for request and response data.
- **Easy to send and receive** between frontend and backend.
- **Easy to debug** using tools like Postman and browser developer tools like swagger(fastapi).

Parse = Read the data and convert it into a usable structure for the program.

5. Headers

A header is additional information sent along with an HTTP request or response. It tells the client or server how to handle the request/response.

They don't usually contain the main business data; they provide **metadata** about the communication.

Common request headers

```
Content-Type: application/json
Authorization: Bearer <token>
Accept: application/json
```

Important headers

- **Content-Type** → tells the server what format the request body uses.
- **Accept** → tells the server what response format the client expects.
- **Authorization** → sends authentication credentials/token.

Example

```
POST /api/employees
Content-Type: application/json
Authorization: Bearer abc123
```

Here:

- **Content-Type:** `application/json` → tells the server that the request body contains JSON.
- **Authorization:** `Bearer abc123` → provides authentication information.

Common Headers

Header	Purpose
Content-Type	Tells the format of the data
Accept	Tells what response format the client wants
Authorization	Sends authentication credentials/token
User-Agent	Identifies the client/application

Request structure

HTTP Request

- Method → POST
- URL → /api/employees
- Headers → Content-Type, Authorization
- Body → JSON employee data

6. URL

A **URL (Uniform Resource Locator)** identifies the resource that the client wants to access.

URL serves as the **unique digital address** used to locate and interact with a specific **resource** over the internet.

Example:

https://example.com/api/employees/101

It can be understood as:

https://example.com/api/employees/101

protocol host resource ID

In REST APIs, URLs generally represent **resources**, such as:

```
/api/employees
/api/departments
/api/products
/api/orders
```

7. Query Parameters

Query parameters are values added to the URL after ?.

They are commonly used for:

- Filtering
- Searching
- Sorting
- Pagination

Example

```
GET /api/employees?department=IT
```

This means:

Get employees whose department is IT.

Multiple parameters:

```
GET /api/employees?department=IT&page=2&limit=10
```

Here:

```
department = IT  
page       = 2  
limit      = 10
```

Query parameters are generally **optional**.

8. Path Parameters

A path parameter is a value included directly in the URL path that is used to identify a specific resource.

A path parameter is a variable part of a URL used to identify a specific resource in a web application or API.

Example:

```
GET /api/employees/101
```

Here **101** is the employee ID.

The API can be designed as:

```
GET /api/employees/{id}
```

For example:

```
/api/employees/101  
/api/employees/205  
/api/employees/500
```

Each request refers to a particular employee.

Query vs Path

/api/employees/101

↑
Path parameter

/api/employees?department=IT

↑
Query parameter

Simple rule:

Path parameter → **Which resource?**

Query parameter → **How do I want the resources?**

9. Request Body

The **request body** contains data sent from the client to the server.

It is mainly used with:

- POST
- PUT
- PATCH

Example

POST /api/employees

Content-Type: application/json

Request body:

```
{
  "name": "Rohit",
  "email": "rohit@example.com",
  "department": "IT",
  "salary": 50000
}
```

The backend receives this data, validates it, and may store it in the database.

10. HTTP Methods

HTTP methods tell the server **what operation the client wants to perform**.

Method	Purpose	Example
GET	Retrieve data	GET /employees

Method	Purpose	Example
POST	Create data	POST /employees
PUT	Replace/update complete resource	PUT /employees/102
PATCH	Partially update resource	PATCH /employees/102
DELETE	Delete resource	DELETE /employees/102

GET – Retrieve Data

Used when the client wants to **read data**.

GET /api/employees

Response:

```
[
  {
    "id": 101,
    "name": "Rohit"
  },
  {
    "id": 102,
    "name": "Amit"
  }
]
```

Another example:

GET /api/employees/101

This retrieves one specific employee.

POST – Create Data

Used to **create a new resource**.

POST /api/employees

Body:

```
{
  "name": "Rohit",
  "email": "rohit@example.com",
  "department": "IT"
}
```

Successful response:

201 Created

```
{
  "id": 101,
  "name": "Rohit",
  "email": "rohit@example.com",
}
```

```
}  "department": "IT"
}
```

PUT – Complete Update

PUT is generally used when you want to **replace/update the complete resource**.

PUT /api/employees/101

Body:

```
{
  "name": "Rohit Raj Singh",
  "email": "rohit@example.com",
  "department": "Engineering"
}
```

Think:

PUT = Replace the complete resource representation.

PATCH – Partial Update

PATCH is used when you only want to **change specific fields**.

PATCH /api/employees/101

Body:

```
{
  "department": "Data Engineering"
}
```

Only the department needs to be changed.

PATCH = Change only what is provided.

DELETE – Remove Data

Used to delete a resource.

DELETE /api/employees/101

If the employee exists, the server deletes employee 101.

A successful response may be:

204 No Content

11. HTTP Status Codes

Status codes tell the client **what happened after processing the request**.

2xx – Success

200 OK

Request successfully processed.

Example:

```
GET /api/employees  
→ 200 OK
```

201 Created

A new resource was successfully created.

```
POST /api/employees  
→ 201 Created
```

204 No Content

Request succeeded, but there is no response body.

Common example:

```
DELETE /api/employees/101  
→ 204 No Content
```

4xx – Client Errors

These usually mean something is wrong with the request or the client's permissions.

400 Bad Request

The request is invalid.

401 Unauthorized

Authentication is missing or invalid.

403 Forbidden

The user is authenticated but doesn't have permission.

404 Not Found

The requested resource doesn't exist.

Example:

```
GET /api/employees/9999  
→ 404 Not Found
```

422 Unprocessable Entity

5xx – Server Errors

These indicate a problem on the server.

500 Internal Server Error

For example, an unexpected backend exception occurs while processing the request.

12. REST API Example

For an **Employee Management API**.

Get all employees

```
GET /api/employees
```

Get one employee

```
GET /api/employees/101
```

Create employee

```
POST /api/employees
```

```
{  
  "name": "Rohit",  
  "email": "rohit@example.com",  
  "department": "IT"  
}
```

Update employee completely

```
PUT /api/employees/101
```

Update specific field

```
PATCH /api/employees/101
```

```
{  
  "department": "Engineering"  
}
```

Delete employee

DELETE /api/employees/101

Developer View

Whenever we build or consume an API, think about these **5 things**:

1. METHOD → What operation?
2. URL → Which resource?
3. PARAMETERS → Which/filtering data?
4. BODY → What data am I sending?
5. RESPONSE → What did the server return?

The backend then:

```
Receive Request  
  ↓  
Authenticate  
  ↓  
Validate Data  
  ↓  
Business Logic  
  ↓  
Database Operation  
  ↓  
Return Response
```

Employee REST API Design

1. GET /employees

Purpose: Get all employees.

Request JSON: Not required.

Response JSON:

```
{  
  "employees": [  
    {
```

```
    "id": 1,  
    "name": "Rohit",  
    "email": "rohit@example.com",  
    "department": "IT"  
  },  
  {  
    "id": 2,  
    "name": "Amit",  
    "email": "amit@example.com",  
    "department": "HR"  
  },  
  {  
    "id": 3,  
    "name": "Priti",  
    "email": "priti@example.com",  
    "department": "Finance"  
  }  
]  
}
```

Success: 200 OK

Errors: 500 Internal Server Error

Validation: No request-body validation required.

2. GET /employees/{id}

Purpose: Get a specific employee.

Example:

GET /employees/1

Request JSON: Not required.

Response JSON:

```
{  
  "id": 1,  
  "name": "Rohit",  
  "email": "rohit@example.com",  
  "department": "IT"  
}
```

Success: 200 OK

Errors:

- 404 Not Found → Employee doesn't exist.
- 422 Unprocessable Entity → Invalid id.

Validation: id must be a valid integer.

3. POST /employees

Purpose: Create a new employee.

Request JSON:

```
{  
  "name": "Rohit",  
  "email": "rohit@example.com",  
  "department": "IT"  
}
```

Response JSON:

```
{  
  "id": 1,  
  "name": "Rohit",  
  "email": "rohit@example.com",  
  "department": "IT"  
}
```

Success: 201 Created

Errors:

- 400 Bad Request → Invalid request.
- 409 Conflict → Email already exists.
- 422 Unprocessable Entity → Validation error.

Validation:

- name is required.
- email is required and must be valid.
- department is required.
- Email should be unique.

4. PUT /employees/{id}

Purpose: Completely update an existing employee.

Example:

PUT /employees/1

Request JSON:

```
{  
  "name": "Rohit Singh",  
  "email": "rohit@example.com",  
  "department": "Data Engineering"  
}
```

Response JSON:

```
{  
  "id": 1,  
  "name": "Rohit Raj Singh",  
  "email": "rohit@example.com",  
  "department": "Data Engineering"  
}
```

Success: 200 OK

Errors:

- 404 Not Found → Employee doesn't exist.
- 409 Conflict → Email conflicts with another employee.
- 422 Unprocessable Entity → Invalid data.

Validation:

- id must be a valid integer.
- Required fields must be provided.
- Email must have a valid format.

5. DELETE /employees/{id}

Purpose: Delete a specific employee.

Example:

DELETE /employees/1

Request JSON: Not required.

Response JSON: Usually no body.

Success: 204 No Content

Errors:

- 404 Not Found → Employee doesn't exist.
- 422 Unprocessable Entity → Invalid id.

Validation: id must be a valid integer.

Final API Table:

Method	Endpoint	Request JSON	Success	Main Errors
GET	/employees	None	200	500
GET	/employees/{id}	None	200	404, 422
POST	/employees	Required	201	400, 409, 422
PUT	/employees/{id}	Required	200	404, 409, 422
DELETE	/employees/{id}	None	204	404, 422

Errors:

Status	Meaning	Simple Example
400	Bad request	Malformed/invalid request
404	Not found	Employee doesn't exist
409	Conflict	Duplicate email
422	Validation error	Invalid email / missing required field
500	Server error	Database/server failure